Jain Pay — Unified AI Subsystem & Next-Gen Chat Architecture PRD

Product: Jain Pay (Native Android — Kotlin, Jetpack Compose, Material 3, Room, Firebase Auth, Firestore, Cloud Functions)
Date: 25 October 2026
Scope: Consolidated specification of (1) all previously planned AI features from the Jain Pay Master PRD, and (2) newly proposed next-generation AI chat and payment features designed to surpass Google Pay, PhonePe, Paytm, and CRED.

0. Core Architectural & Safety Guardrails (Non-Negotiable)

1. **AI Stages, Deterministic Backend Executes:** No AI model, prompt, or tool call may ever directly execute a money transfer (`createPayment`), issue a refund (`processRefund`), alter an account balance (`adjustDemoBalance`), or modify a security PIN (`updatePins`). Every financial mutation must route through the deterministic Jetpack Compose Review Screen → PIN Verification → Firebase Cloud Functions atomic ledger transaction flow.
2. **Trusted Server-Side Proxy:** Hosted AI models (such as Gemini 2.5 Flash / Vision) must be called exclusively through authenticated Firebase Cloud Functions. No API keys or privileged credentials may be embedded in the Android APK.
3. **Deterministic Math Only:** All currency totals, bill splits, tax proportions, budget thresholds, and cash-flow balances must be computed by deterministic Kotlin or Node.js backend code using currency-safe decimal arithmetic—never by LLM mental math.
4. **Strict Redaction & Privacy:** Never transmit raw PINs, PIN hashes/salts, OTPs, Firebase Auth tokens, App Check tokens, or full unmasked contact lists to any AI model.
5. **Separation of AI Tracks:** Track A (the developer's local coding LLM for repository maintenance) remains completely separate from Track B (user-facing in-app AI features inside Jain Pay).

SECTION 1: Baseline Planned AI Features (From Original Master PRD)

This section consolidates the eight foundational AI modules and five Google AI Studio subsystem specifications already planned for Jain Pay.

1.1 AI-01: Deterministic Spending Insights & Cash-Flow Summaries (Priority: P1)

- **Purpose:** Summarize verified user spending across time ranges, categories, and counterparties.

- **Capabilities:**
    - Week-over-week and month-over-month spending comparisons.
    - Top spending categories and frequent merchants/counterparties.
    - Plain-language explanation of unusual spend spikes grounded in deterministic aggregates.
- **Guardrails:** Totals are computed in Kotlin/Cloud Functions before passing a redacted summary to the model. Explicitly states date coverage and clarifies that insights reflect Jain Pay ledger activity only.

1.2 AI-02: Natural-Language Transaction Search (Priority: P1)

- **Purpose:** Let users search their activity history conversationally (e.g., "Show payments above ₹500 last month", "How much did I spend on electricity bills this quarter?", "Show failed payments from this week").
- **Capabilities:**
    - Translates natural language (English, Hindi, Hinglish) into a constrained, schema-validated filter object.
    - Renders matching records directly inside the existing `ActivityHistoryScreen` and receipt detail views.
- **Guardrails:** Never generates raw SQL or arbitrary Firestore query paths from free-form text; asks for clarification when a query is ambiguous.

1.3 AI-03: Smart Budgets & Proactive Spending Alerts (Priority: P2)

- **Purpose:** Empower users to configure category-level monthly budgets and receive proactive threshold alerts.
- **Capabilities:**
    - User-defined category caps with real-time progress tracking.
    - Transparent pace estimates prior to month-end based on historical velocity.
- **Guardrails:** Requires user opt-in; never freezes accounts or blocks payments automatically when a budget threshold is crossed.

1.4 AI-04: Predictive Cashflow & Recurring Payment Detection Engine (Priority: P2)

- **Purpose:** Identify recurring obligations (mobile recharges, rent transfers, electricity/utility bills, subscriptions) and forecast potential balance shortfalls up to 48 hours in advance.
- **Capabilities:**
    - Detects candidate recurring payment cycles from historical ledger records.
    - Prompts users to confirm, edit, snooze, or disable reminders.
    - Projects short-term cash depletion against upcoming expected bills.
- **Guardrails:** Begins with transparent deterministic recurrence logic before introducing time-series models; reminders never trigger autonomous auto-pay.

1.5 AI-05: Jain Pay Voice & Text Co-Pilot & Core Tool Contracts (Priority: P2)

- **Purpose:** A conversational assistant supporting English, Hindi, and Hinglish that explains app features, answers transaction status questions, and stages actions via strict server/client tool contracts.
- **Core Tool Contracts:**
    - `stagePeerPayment(recipientIdentifier, amount, paymentNote?)`: Validates and resolves a contact/VPA through deterministic app logic and pre-fills the standard payment review screen. Forbidden from calling `createPayment` or bypassing PIN entry.
    - `queryFinancialLedger(timeRangeDays, transactionType?, category?)`: Queries authorized user records (`ALL`, `SEND`, `RECEIVE`, `BILL_PAYMENT`, `RECHARGE`) via a deterministic query service and returns redacted summaries.
    - `fetchBillDueStatus(billerCategory, consumerNumber)`: Queries an authorized biller integration (`ELECTRICITY`, `WATER`, `GAS`, `BROADBAND`, `FASTAG`) or clearly reports sandbox/unavailable status without fabricating bill amounts.

1.6 AI-06: Multimodal Bill, Receipt & QR Assistance (Priority: P2/P3)

- **Purpose:** Use CameraX and server-proxied Gemini 2.5 Flash Vision to extract structured fields from physical utility bills or receipts and explain QR payloads.
- **Structured Output Contract:** Extracts `billerName`, `consumerId`, `dueDate`, `totalAmountDue`, `billPeriod`, and `confidenceScore`.
- **Guardrails:** All OCR outputs are treated as untrusted suggestions displayed in an editable confirmation form; never auto-submits a bill payment. QR codes remain validated by deterministic `UPIParser` rules (`pa`, `pn`, `mc`, `tr`, `cu`, `am`).

1.7 AI-07: Behavioral Fraud & Anomaly Detection Signals (Priority: P3 — High Governance)

- **Purpose:** Surface cautionary risk signals prior to payment authorization using minimally invasive, privacy-reviewed telemetry (e.g., 15-minute trailing transaction velocity, unusual transaction hour relative to user baseline, and verified app/device attestation).
- **Risk Tiering:**
    - **Low Risk:** Standard Payment PIN flow.
    - **Medium Risk:** Step-up biometric or secondary confirmation prompt.
    - **Critical Risk:** Deterministic soft-hold with an explainable warning banner and explicit verification challenge.
- **Guardrails:** Never logs raw PIN digits or keystroke secrets; an LLM never autonomously freezes accounts, approves/declines transactions, or files reports.

1.8 AI-08: Contextual-Bandit Rewards Optimization & Smart Discovery (Priority: P3)

- **Purpose:** Guide users to relevant app workflows (Recharge, Bills, Receive QR, Activity) and optimize reward/voucher category relevance (LinUCB contextual bandit) within strict merchant margin budgets.
- **Guardrails:** All scratch-card generation (e.g., transactions > ₹50) and cashback ledger credits remain strictly deterministic, idempotent, and server-authoritative inside Cloud Functions.

SECTION 2: Newly Proposed Next-Gen AI Chat & Payment Features (Competitive Differentiators)

Existing Indian payment apps (Google Pay, PhonePe, Paytm, CRED) treat P2P chat as a passive receipt log with a basic equal-split calculator. The following seven features transform Jain Pay's chat into an interactive, multimodal financial command deck while preserving every security boundary in Section 0.

2.1 NEXT-01: Multimodal "Snap-and-Split" Itemized Bill Agent in P2P & Group Chat

- **Market Gap:** Competitors force users to manually calculate itemized restaurant bills, proportional GST, service charges, and discounts outside the app.
- **User Experience:** Inside a P2P or group chat thread, a user snaps a photo of a dining bill or uploads a Zomato/Swiggy/Blinkit invoice PDF/screenshot and types or speaks in Hinglish: *"Aman didn't have the cold coffee, split the rest equally between the three of us."*
- **Technical Flow & Tool Contract (`parseAndSplitReceipt`):**
    a. Gemini Vision extracts line items (`itemName`, `quantity`, `unitPrice`, `taxAmount`, `discountAmount`).
    b. Deterministic backend code verifies that line items + taxes - discounts equal the grand total, then calculates each participant's exact share down to the paisa.
    c. Renders an interactive **Item-Claim Compose Card** inside the chat thread where participants tap their items and press **"Pay My Share"**, which stages a standard `stagePeerPayment` request into the PIN-protected payment flow.

2.2 NEXT-02: Conversational "Net-Zero Khata" (Micro-IOU Running Tab Inside P2P Chat)

- **Market Gap:** Friends, hostelmates, and flatmates make dozens of ₹20–₹150 micro-transactions weekly (chai, auto rides, shared groceries), cluttering bank statements and failing during bank server downtimes.

- **User Experience:** Users type naturally in their 1-on-1 P2P chat: *"Paid ₹120 for auto, split 50-50"* or *"Deduct the ₹40 Maggi I owed you yesterday."*
- **Technical Flow & Tool Contracts (`proposeChatKhataEntry`, `stageKhataSettlement`):**
    a. The AI parses the conversational message into a structured IOU entry (`amount`, `splitMode`, `note`, `direction`) stored in a non-wallet Firestore subcollection (`/chats/{chatId}/khataEntries/{entryId}`).
    b. Once the counterparty taps **"Acknowledge"** on the chat bubble, the deterministic net running balance header updates at the top of the chat (e.g., *"Net Khata: You owe Rohan ₹190"*).
    c. At the end of the week or month, tapping **"Settle Net Balance"** stages a single real payment (`stagePeerPayment`) for the exact net difference through the standard PIN flow.

2.3 NEXT-03: Pre-Flight Scam & Social Engineering Shield in Chat & Scanner

- **Market Gap:** GPay and PhonePe display generic static warnings that users ignore during sophisticated social-engineering scams (fake OLX/marketplace buyers sending "Receive Money" QR codes, fake electricity disconnection SMS, or urgent family impersonation messages).
- **User Experience:** A user forwards a suspicious WhatsApp text/screenshot into Jain Pay Co-Pilot—or scans an unfamiliar QR code—and gets an instant, explainable safety check before entering their PIN.
- **Technical Flow & Tool Contract (`inspectPaymentSafety`):**
    a. Deterministic rules inspect the QR payload (`pa`, `pn`, `am`, `mc`) and counterparty history (first-time recipient, high amount, collect/debit mismatch).
    b. The AI evaluates accompanying message text/screenshots for social-engineering urgency cues (e.g., *"Scan this QR and enter PIN to receive ₹5,000 refund"*).
    c. Displays a high-contrast **Explainable Threat Card** in English/Hinglish highlighting the exact contradiction (*"Warning: Entering your PIN on a QR scan ALWAYS sends money out of your account; it never receives money"*), requiring an explicit safety acknowledgment before proceeding to the review screen.

2.4 NEXT-04: Universal "Any-Screenshot-to-Staged-Payment" Intent Engine

- **Market Gap:** When users receive an event poster with a UPI ID, a handwritten vendor bill on WhatsApp, or a text message with a phone number and amount, they must manually switch apps and type details from memory.
- **User Experience:** Users share any image or text snippet from Android's native Share Sheet directly to **"Jain Pay Co-Pilot"**.
- **Technical Flow & Tool Contract (`extractPaymentIntentFromMedia`):**
    a. Extracts candidate `recipientVpaOrPhone`, `amount`, and `paymentNote` from the shared screenshot or text.

b. Deterministically cross-checks `queryFinancialLedger` over the past 7 days to warn the user if an identical amount was already paid to that recipient (preventing accidental double payments).
c. Opens a pre-populated Staged Payment Card inside the chat where one tap takes the user to `SendMoneyScreen` / `PaymentFlowScreens`.

2.5 NEXT-05: Interactive "What-If" Safe-to-Spend & Cashflow Runway Simulator

- **Market Gap:** Payment apps only show historical spend after money is gone; they do not help users decide whether an impulse purchase is safe today.
- **User Experience:** A user asks the Co-Pilot in chat: *"Can I afford ₹3,500 headphones today without messing up my month-end bills?"*
- **Technical Flow & Tool Contract (`simulateSafeToSpend`):**
    a. Deterministic backend service retrieves the user's current balance (once unlocked/authorized), subtracts predicted upcoming recurring bills from AI-04 (e.g., electricity bill, mobile recharge) and remaining active category budgets from AI-03.
    b. Computes `safeToSpendBeforePurchase` and `projectedRunwayAfterPurchase`.
    c. Renders an interactive **Safe-to-Spend Runway Card** in the chat showing the exact impact of the hypothetical purchase before the user spends a rupee.

2.6 NEXT-06: Milestone-Staged "Trust Cards" for Conditional P2P Deals

- **Market Gap:** Peer-to-peer transactions for freelance work, college project supplies, second-hand electronics, or shared deposits have no structured tracking inside payment chats.
- **User Experience:** Inside a P2P chat, a user tells the AI: *"Create a 2-step payment tracker with Vikram for the monitor: ₹1,000 advance today, ₹3,000 when I pick it up on Saturday."*
- **Technical Flow & Tool Contract (`createMilestoneTrustCard`):**
    a. Creates a non-custodial, stateful **Milestone Deal Card** inside the shared chat thread (`CREATED` → `ADVANCE_PAID` → `DELIVERY_CONFIRMED` → `SETTLED`), without holding funds in unauthorized escrow.
    b. Milestone 1 surfaces a one-tap **"Pay ₹1,000 Advance"** button that routes through the standard PIN payment flow and links the resulting authoritative `transactionId` to the card.
    c. Once both users tap **"Confirm Handover"** on Saturday, Milestone 2 unlocks the staged **"Pay ₹3,000 Final Balance"** button.

2.7 NEXT-07: Conversational Transaction Diagnostician & Smart Receipt Verifier

- **Market Gap:** When a payment is pending, timed out, or failed, users panic and retry immediately (risking double debits) or struggle to interpret bank reference numbers.

- **User Experience:** Tapping *"Ask Co-Pilot about this transaction"* on any row in `ActivityHistoryScreen` opens a contextual chat card that explains the exact state of that transaction and safe next steps.
- **Technical Flow & Tool Contract (`diagnoseTransactionState`):**
    a. Calls the authoritative `getPaymentStatus` Cloud Function and checks `IdempotencyManager` state for the specific `txnId`.
    b. Explains in clear English/Hinglish whether the money left the sender's ledger, whether the idempotency lock is still active, and whether it is safe to retry—completely eliminating blind duplicate retries.

SECTION 3: Unified Tool Registry & Architecture Matrix

Tool / Function Name	Category	Execution Layer	Allowed Output / Side Effect
`stagePeerPayment`	Baseline (AI-05)	Cloud Function Proxy → Client ViewModel	Populates staged `PaymentRequest`; routes to Review Screen
`queryFinancialLedger`	Baseline (AI-02/05)	Cloud Function Proxy → Firestore Read	Returns deterministic aggregates & redacted transaction list
`fetchBillDueStatus`	Baseline (AI-05)	Cloud Function Proxy → Biller Sandbox	Returns bill status or explicit sandbox indicator
`parseAndSplitReceipt`	New (NEXT-01)	Gemini Vision Proxy + Deterministic Math	Renders interactive Item-Claim Split Card in chat
`proposeChatKhataEntry`	New (NEXT-02)	Cloud Function → `/chats/{id}/khataEntries`	Creates non-wallet IOU entry awaiting peer acknowledgment
`stageKhataSettlement`	New (NEXT-02)	Deterministic Net Calculator → `stagePeerPayment`	Stages a single payment for the net IOU balance
`inspectPaymentSafety`	New (NEXT-03)	Deterministic QR Rules + Scam Classifier	Renders Explainable Threat Card prior to Review/PIN
`extractPaymentIntentFromMedia`	New (NEXT-04)	Gemini Vision Proxy + Duplicate Check	Stages payment card from shared screenshot/poster
`simulateSafeToSpend`	New (NEXT-05)	Deterministic Cashflow Service	Renders Safe-to-Spend Runway Bar in chat

Tool / Function Name	Category	Execution Layer	Allowed Output / Side Effect
`createMilestoneTrustCard`	New (NEXT-06)	Cloud Function → `/chats/{id}/milestones`	Renders stateful multi-step P2P deal tracker in chat
`diagnoseTransactionState`	New (NEXT-07)	`getPaymentStatus` + Idempotency Audit	Explains exact transaction status & safe retry guidance

SECTION 4: Staged Implementation Roadmap

1. **Stage 0 (Prerequisite Stabilization):** Verify active runtime wiring of `DESIGN.md` ("Harness.io — Style Reference" dark-first command deck tokens), resolve the `BalancePinDialog` signature/call-site audit, unify PIN hashing across client and `functions/index.js`, and complete local Firebase CLI deployment.
2. **Stage 1 (Core Co-Pilot & Ledger Search):** Implement the server-side `askJainPayCopilot` Cloud Function proxy with `queryFinancialLedger`, `stagePeerPayment`, and `diagnoseTransactionState` (AI-01, AI-02, AI-05, NEXT-07).
3. **Stage 2 (P2P Chat Intelligence):** Launch **Conversational Net-Zero Khata** (NEXT-02) and **Milestone-Staged Trust Cards** (NEXT-06) inside P2P chat threads.
4. **Stage 3 (Multimodal Vision & Safety Shield):** Enable **Snap-and-Split Receipt Agent** (NEXT-01), **Any-Screenshot-to-Staged-Payment** (NEXT-04), **Bill OCR** (AI-06), and **Pre-Flight Scam Shield** (NEXT-03).
5. **Stage 4 (Predictive & Behavioral Intelligence):** Roll out **Safe-to-Spend Cashflow Simulator** (AI-03, AI-04, NEXT-05), **Anomaly Signals** (AI-07), and **Contextual Rewards** (AI-08).